ML Model Deployment on EC2 and Access it

Task 1 - Display different classification models' accuracy_score, precision_score, recall_score, and f1_score and display it in a tabular form

Model Performance Evaluation:

	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.95855	0.956009	0.95855	0.955330
1	Decision Tree	0.95505	0.954850	0.95505	0.954948
2	Random Forest	0.96820	0.966865	0.96820	0.966417
3	Gradient Boosting	0.97245	0.972937	0.97245	0.970133
4	SVM	0.95960	0.958052	0.95960	0.955490
5	KNN	0.95920	0.957349	0.95920	0.955232
6	Naive Bayes	0.90720	0.920187	0.90720	0.912697

ML Model Deployment on EC2 and Access it

Task 2 - Create an EC2 instance

Description - *required* | Info

launch-wizard-3 created 2025-11-30T14:57:02.885Z

Inbound Security Group Rules

▼ Security group rule 1 (TCP, 22, 0.0.0.0/0) — Remove

Type	Protocol	Port range
ssh	TCP	22

Source type	Source	Description - *optional*
Anywhere	Add CIDR, prefix list or security group 0.0.0.0/0	e.g. SSH for admin desktop

▼ Security group rule 2 (TCP, 8000, 0.0.0.0/0, FastAPI / Uvicorn access) — Remove

Type	Protocol	Port range
Custom TCP	TCP	8000

Source type	Source	Description - *optional*
Anywhere	Add CIDR, prefix list or security group 0.0.0.0/0	FastAPI / Uvicorn access

⚠ Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

Add security group rule

ML Model Deployment on EC2 and Access it

Task 3 - Connect to EC2 from your laptop using **Command Prompt**

scp -i mykey.pem app.py requirements.txt diabetes_model.pkl ubuntu@<EC2_IPv4>:/home/ubuntu/

```
C:\Personal\AUPP + FHSU\AUPP 2025\AUPP Fall 2025\CLD 375 (Cloud Machine Learning Foundation and Data Engineering)\A
ssignment\ML Model Deployment on EC2 and Access it>scp -i mykey2.pem app.py requirements.txt diabetes_model.pkl ubu
ntu@98.84.29.40:/home/ubuntu/
app.py                                                                    100% 1832     6.3KB/s   00:00
requirements.txt                                                          100%   57     0.2KB/s   00:00
diabetes_model.pkl                                                        100%  129KB 124.6KB/s   00:01
```

ssh -i "mykey.pem" ubuntu@<EC2_IPv4>

```
Last login: Sun Nov 30 14:32:31 2025 from 175.100.10.210
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-64-209:~$
```

ML Model Deployment on EC2 and Access it

Task 4 - Install necessary updates/software to run your API

1). update + install python3 venv

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3-pip python3-venv git
```

2). create venv and install requirements

```
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

requirements.txt

necessary libraries:

- fastapi
- uvicorn[standard]
- boto3
- scikit-learn==1.6.1
- pydantic

ML Model Deployment on EC2 and Access it

Task 5 - Convert your model to a pickle file

```
import pickle
from sklearn.ensemble import GradientBoostingClassifier

with open("diabetes_model.pkl", "wb") as f:
    pickle.dump(gb_model, f)
print("Saved diabetes_model.pkl")
```

```
Saved diabetes_model.pkl
```

ML Model Deployment on EC2 and Access it

Task 6 - Create your API file

```python
# app.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import pickle, boto3, json, uuid, os
from datetime import datetime, timezone

# ========== CONFIG ==========
MODEL_PATH = "diabetes_model.pkl"
S3_BUCKET = "orpf-prediction-bucket"
AWS_REGION = "us-east-1"
# ============================

class Person(BaseModel):
    # put feature names exactly as the model expects
    age: float
    bmi: float
    blood_glucose_level: float
    HbA1c_level: float
    hypertension: float
    heart_disease: float

app = FastAPI()

# Load model once on startup
with open(MODEL_PATH, "rb") as f:
    model = pickle.load(f)

# derive class info for human-readable output
classes = list(getattr(model, "classes_", []))
if set(classes) == {0, 1}:
    CLASS_MAP = {0: "non-diabetic", 1: "diabetic"}
else:
    CLASS_MAP = {c: str(c) for c in classes}

# Comment out the below line if not saving to S3
s3 = boto3.client("s3", region_name=AWS_REGION)

def upload_json_to_s3(obj: dict, key_prefix="logs/"):
    key = f"{key_prefix}{datetime.now(timezone.utc).isoformat()}_{uuid.uuid4().hex}.json"
    s3.put_object(Bucket=S3_BUCKET, Key=key, Body=json.dumps(obj).encode("utf-8"))
    return key

@app.post("/predict")
def predict(p: Person):
    try:
        # convert p to model input shape
        # Example: if model expects a 2D list
        X = [[p.age, p.bmi, p.blood_glucose_level, p.HbA1c_level, p.hypertension, p.heart_disease]]
        pred = model.predict(X)[0]
        prob = model.predict_proba(X)[0][1] if hasattr(model, "predict_proba") else None # Just class 1 probability
        prob_percentage = round(prob * 100, 2) if prob is not None else None
        label_name = CLASS_MAP.get(pred, str(pred))

        record = {
            "input": p.model_dump(),
            "prediction": int(pred),
            "label": label_name,
            "probability": prob,
            "timestamp": datetime.now(timezone.utc).isoformat()
        }

        upload_key = upload_json_to_s3(record)

        return {"result": int(pred), "label": label_name, "probability": f"{prob_percentage}%", "s3_key": upload_key}
        # return {"result": int(pred), "label": label_name, "probability": f"{prob_percentage}%"}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

ML Model Deployment on EC2 and Access it

Task 7 - Upload/deploy it to the EC2 instance

Task 8 - Run your api inside EC2 instance

Task 9 - Using Postman, send requests to the API

```
ubuntu@ip-172-31-72-1: ~

(venv) ubuntu@ip-172-31-72-1:~$ uvicorn app:app --host 0.0.0.0 --port 8000
INFO:     Started server process [13517]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     203.176.133.182:61458 - "GET /docs HTTP/1.1" 200 OK
INFO:     203.176.133.182:61458 - "GET /openapi.json HTTP/1.1" 200 OK
/home/ubuntu/venv/lib/python3.10/site-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feat
ure names, but GradientBoostingClassifier was fitted with feature names
  warnings.warn(
/home/ubuntu/venv/lib/python3.10/site-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feat
ure names, but GradientBoostingClassifier was fitted with feature names
  warnings.warn(
INFO:     203.176.133.182:61458 - "POST /predict HTTP/1.1" 200 OK
/home/ubuntu/venv/lib/python3.10/site-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feat
ure names, but GradientBoostingClassifier was fitted with feature names
  warnings.warn(
/home/ubuntu/venv/lib/python3.10/site-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feat
ure names, but GradientBoostingClassifier was fitted with feature names
  warnings.warn(
INFO:     203.176.133.182:52237 - "POST /predict HTTP/1.1" 200 OK
```

ML Model Deployment on EC2 and Access it

Task 10 - Display prediction in the postman

Person 1

Input

```
{
  "age": 50,
  "bmi": 32.35,
  "blood_glucose_level": 80,
  "HbA1c_level": 6.23,
  "hypertension": 1,
  "heart_disease": 1
}
```

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://3.238.242.245:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "age": 50,
  "bmi": 32.35,
  "blood_glucose_level": 80,
  "HbA1c_level": 6.23,
  "hypertension": 1,
  "heart_disease": 1
}'
```

Request URL

```
http://3.238.242.245:8000/predict
```

Server response

Code | Details

200

Response body

```
{
  "result": 0,
  "label": "non-diabetic",
  "probability": "99.75%",
  "s3_key": "logs/2025-12-02T15:20:06.124245+00:00_2941382e5e7c433b8215212ed8ef92ac.json"
}
```

Output

Download

Response headers

```
content-length: 141
content-type: application/json
date: Tue,02 Dec 2025 15:20:05 GMT
server: uvicorn
```

Person 2

Input

```
{
  "age": 30,
  "bmi": 26.15,
  "blood_glucose_level": 90,
  "HbA1c_level": 4.23,
  "hypertension": 0,
  "heart_disease": 0
}
```

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://3.238.242.245:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "age": 30,
  "bmi": 26.15,
  "blood_glucose_level": 90,
  "HbA1c_level": 4.23,
  "hypertension": 0,
  "heart_disease": 0
}'
```

Request URL

```
http://3.238.242.245:8000/predict
```

Server response

Code | Details

200

Response body

```
{
  "result": 0,
  "label": "non-diabetic",
  "probability": "99.78%",
  "s3_key": "logs/2025-12-02T15:23:00.614397+00:00_02f7b48cb4df4cd6b8f985249172944f.json"
}
```

Output

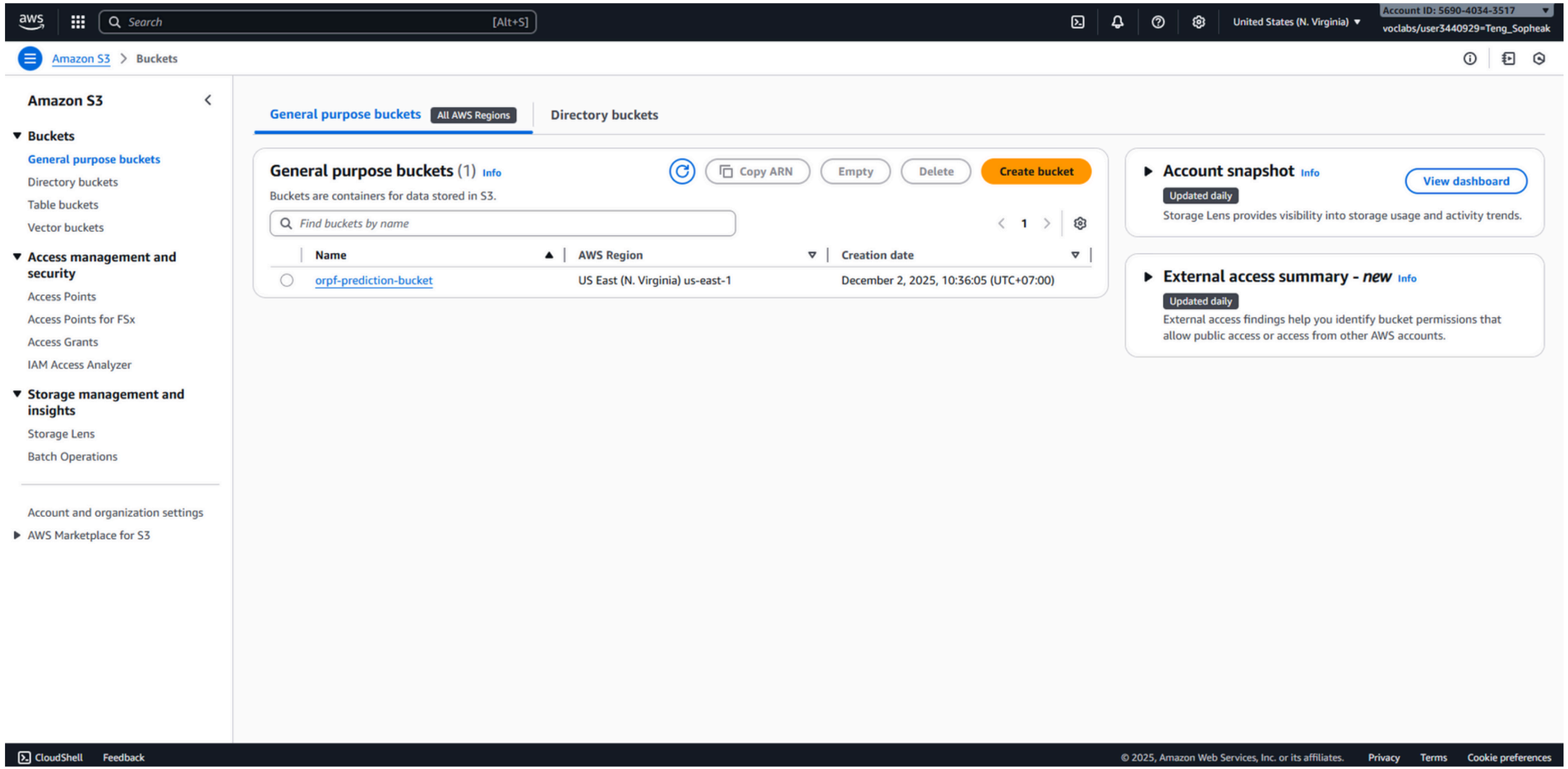
Download

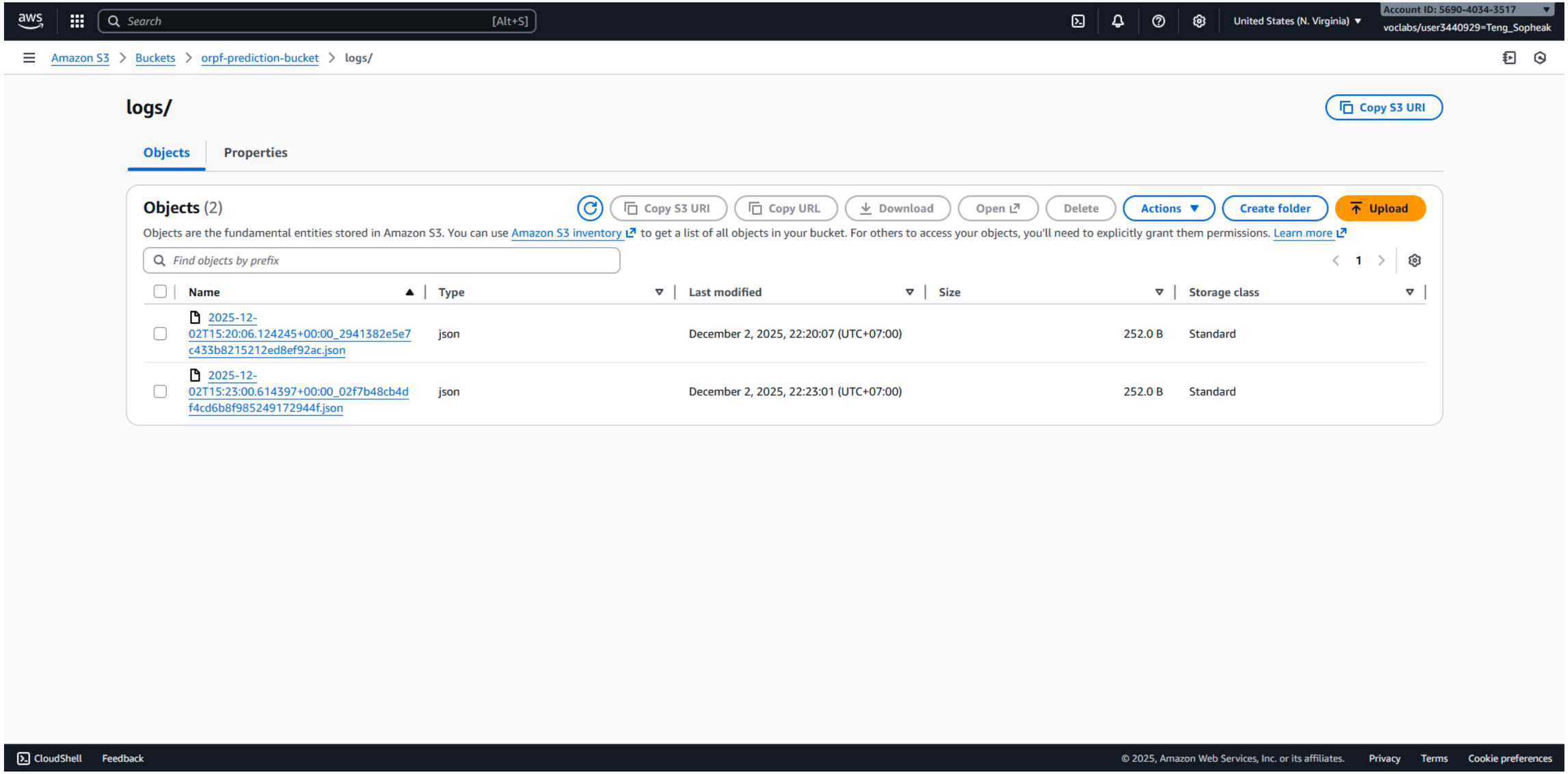
Response headers

```
content-length: 141
content-type: application/json
date: Tue,02 Dec 2025 15:23:00 GMT
server: uvicorn
```

ML Model Deployment on EC2 and Access it

Task 11 - Create an S3 bucket



ML Model Deployment on EC2 and Access it

Task 12 - Try to store the input data and output prediction in the S3 bucket